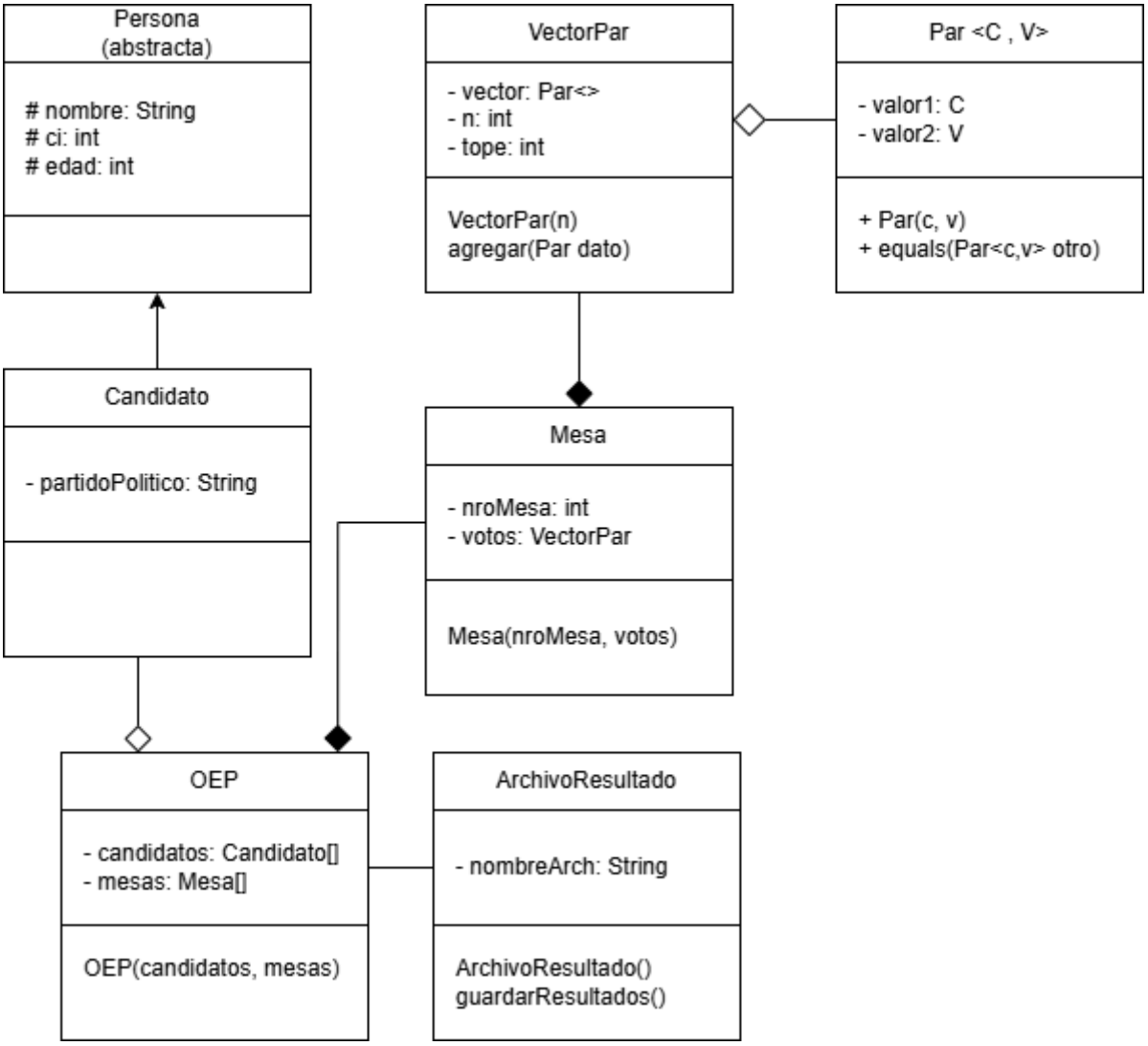
EXAMEN RECUPERATORIO
PROGRAMACIÓN II (INF-121) I/2025
16 DE JUNIO DE 2025

NOMBRES y APELLIDOS:

CI: **PAR:**

Se pide implementar en Python o Java el siguiente diagrama y resolver los siguientes incisos (contexto: votaciones):



(“MNR”, 25)	(“MAS”, 1)	(“NP”, 12)	(“CONDEPA”, 23)	(“ADN”, 11)

a) Mostrar todos lo datos de la OEP

b) Mostrar la cantidad de votos más baja del candidato del partido X

c) Mostrar los candidatos, su partido y la cantidad total de votos que obtuvieron

d) Guardar a/los candidato(s), el partido al que representan y los votos, que obtuvo/obtuvieron

IMPLEMENTACION EN PYTHON

```python
import json

class Persona:
    def __init__(self, nombre, ci, edad):
        self.nombre = nombre
        self.ci = ci
        self.edad = edad

    def mostrar(self):
        print(f"{self.nombre} {self.ci} {self.edad}")

    def getNombre(self):
        return self.nombre


class Candidato(Persona):
    def __init__(self, nombre, ci, edad, partidoPolitico):
        super().__init__(nombre, ci, edad)
        self.partidoPolitico = partidoPolitico

    def mostrar(self):
        print(f"{self.nombre} {self.ci} {self.edad}
{self.partidoPolitico}")

    def getPartidoPolitico(self):
        return self.partidoPolitico
    def getNombre(self):
        return super().getNombre()

class Par:
    def __init__(self, valor1, valor2):
        self.valor1 = valor1
```

```
        self.valor2 = valor2

    def mostrar(self):
        print(f"{self.valor1} {self.valor2}")

    def getValor1(self):
        return self.valor1

    def getValor2(self):
        return self.valor2

    def __lt__(self, otro):
        return self.valor2 > otro.getValor2()
    def __add__(self, otro):
        return self.valor2 + otro.getValor2()



class VectorPar:
    def __init__(self, n):
        self.vector = []
        self. n = n
        self.tope = 0

    def agregar(self, par:Par):
        if(self.tope < self.n):
            self.vector.append(par)
            self.tope += 1

    def mostrar(self):
        for par in self.vector:
            par.mostrar()

class Mesa:
```

```
        def __init__(self, nroMesa, votos: VectorPar):
            self.nroMesa = nroMesa
            self.votos = votos
        def mostrar(self):
            print(f"Mesa {self.nroMesa}")
            self.votos.mostrar()

        def getVotos(self):
            return self.votos

    class OEP:
        def __init__(self, candidatos, mesas):
            self.candidatos = candidatos
            self.mesas = mesas

        def mostrar(self):
            print("Candidatos:")
            for candidato in self.candidatos:
                candidato.mostrar()

            print("Mesas:")
            for mesa in self.mesas:
                mesa.mostrar()

        def formatoJson(self):
            data = {}
            i = 1
            for c in self.candidatos:
                data[f"candidato {i}"] ={
                    "nombre": c.getNombre(),
                    "partido": c.getPartidoPolitico(),
                    "votos":
self.conseguirVotosCandidatoX(c.getNombre())
                }
```

```
            i+=1
        return data


class ArchivoResultado:
    def __init__(self, nombreArchivo):
        self.nombreArchivo = nombreArchivo

    def guardar(self, oep: OEP):
        with open(self.nombreArchivo, 'w') as file:
            data = oep.formatoJson()
            json.dump(data, file, indent=4)


c1 = Candidato("Candidato 1", "12345678", 45, "MNR")
c2 = Candidato("Candidato 2", "87654321", 50, "MAS")
c3 = Candidato("Candidato 3", "11223344", 40, "NP")
c4 = Candidato("Candidato 4", "44332211", 55, "CONDEPA")
c5 = Candidato("Candidato 5", "55667788", 60, "ADN")

v1 = VectorPar(10)
v1.agregar(Par("MNR", 24))
v1.agregar(Par("MAS", 30))
v1.agregar(Par("NP", 20))
v1.agregar(Par("CONDEPA", 15))
v1.agregar(Par("ADN", 10))

v2 = VectorPar(10)
v2.agregar(Par("MNR", 20))
v2.agregar(Par("MAS", 25))
v2.agregar(Par("NP", 30))
v2.agregar(Par("CONDEPA", 10))

v3 = VectorPar(10)
```

```
    v3.agregar(Par("MNR", 15))
    v3.agregar(Par("MAS", 20))
    v3.agregar(Par("NP", 25))
    v3.agregar(Par("CONDEPA", 30))
    v3.agregar(Par("ADN", 10))

    m1 = Mesa(1, v1)
    m2 = Mesa(2, v2)
    m3 = Mesa(3, v3)

    oep = OEP([c1, c2, c3, c4, c5], [m1, m2, m3])
```

IMPLEMENTACION EN JAVA

```
import java.io.FileWriter;
import java.io.IOException;
import java.util.ArrayList;
import java.util.List;

// --- Persona Class ---
class Persona {
    protected String nombre;
    protected String ci;
    protected int edad;

    public Persona(String nombre, String ci, int edad) {
        this.nombre = nombre;
        this.ci = ci;
        this.edad = edad;
    }

    public void mostrar() {
        System.out.println(this.nombre + " " + this.ci + " " +
this.edad);
    }

    public String getNombre() {
        return this.nombre;
    }
}

// --- Candidato Class ---
class Candidato extends Persona {
    private String partidoPolitico;
```

```java
    public Candidato(String nombre, String ci, int edad, String
partidoPolitico) {
        super(nombre, ci, edad);
        this.partidoPolitico = partidoPolitico;
    }

    @Override
    public void mostrar() {
        System.out.println(this.getNombre() + " " + this.ci + " " +
this.edad + " " + this.partidoPolitico);
    }

    public String getPartidoPolitico() {
        return this.partidoPolitico;
    }
}

// --- Par Class ---
class Par implements Comparable<Par> {
    private String valor1;
    private int valor2;

    public Par(String valor1, int valor2) {
        this.valor1 = valor1;
        this.valor2 = valor2;
    }

    public void mostrar() {
        System.out.println(this.valor1 + " " + this.valor2);
    }

    public String getValor1() {
        return this.valor1;
    }
```

```java
    public int getValor2() {
        return this.valor2;
    }

    @Override
    public int compareTo(Par otro) {
        return Integer.compare(otro.getValor2(), this.valor2);
    }

    public int add(Par otro) {
        return this.valor2 + otro.getValor2();
    }
}

// --- VectorPar Class ---
class VectorPar {
    private List<Par> vector;
    private int n;
    private int tope;

    public VectorPar(int n) {
        this.vector = new ArrayList<>();
        this.n = n;
        this.tope = 0;
    }

    public void agregar(Par par) {
        if (this.tope < this.n) {
            this.vector.add(par);
            this.tope++;
        }
    }
```

```java
    public void mostrar() {
        for (Par par : this.vector) {
            par.mostrar();
        }
    }

    public Par buscarMenor(String valor1) {
        Par mini = new Par("", 999); // Initialize with a high value
        for (Par v : this.vector) {
            if (v.getValor1().equals(valor1)) {
                if (mini.getValor2() > v.getValor2()) {
                    mini = v;
                }
            }
        }
        return mini;
    }

    public int conseguirVotos(String partido) {
        for (Par v : this.vector) {
            if (v.getValor1().equals(partido)) {
                return v.getValor2();
            }
        }
        return 0;
    }
}

// --- Mesa Class ---
class Mesa {
    private int nroMesa;
    private VectorPar votos;

    public Mesa(int nroMesa, VectorPar votos) {
```

```java
        this.nroMesa = nroMesa;
        this.votos = votos;
    }

    public void mostrar() {
        System.out.println("Mesa " + this.nroMesa);
        this.votos.mostrar();
    }

    public VectorPar getVotos() {
        return this.votos;
    }
}

// --- OEP Class ---
class OEP {
    private List<Candidato> candidatos;
    private List<Mesa> mesas;

    public OEP(List<Candidato> candidatos, List<Mesa> mesas) {
        this.candidatos = candidatos;
        this.mesas = mesas;
    }

    public void mostrar() {
        System.out.println("Candidatos:");
        for (Candidato candidato : this.candidatos) {
            candidato.mostrar();
        }

        System.out.println("\nMesas:");
        for (Mesa mesa : this.mesas) {
            mesa.mostrar();
        }
```

```
    }
}

// --- ArchivoResultado Class ---
class ArchivoResultado {
    private String nombreArchivo;

    public ArchivoResultado(String nombreArchivo) {
        this.nombreArchivo = nombreArchivo;
    }

    public void guardar(OEP oep) {
        try (FileWriter fileWriter = new
FileWriter(this.nombreArchivo)) {
            String data = oep.formatoTexto();
            fileWriter.write(data);
            System.out.println("Datos guardados exitosamente en " +
this.nombreArchivo);
        } catch (IOException e) {
            System.err.println("Error al guardar el archivo: " +
e.getMessage());
        }
    }
}

// --- Main Class for execution ---
public class solucion {
    public static void main(String[] args) {
        Candidato c1 = new Candidato("Candidato 1", "12345678", 45,
"MNR");
        Candidato c2 = new Candidato("Candidato 2", "87654321", 50,
"MAS");
        Candidato c3 = new Candidato("Candidato 3", "11223344", 40,
"NP");
```

```
        Candidato c4 = new Candidato("Candidato 4", "44332211", 55,
"CONDEPA");
        Candidato c5 = new Candidato("Candidato 5", "55667788", 60,
"ADN");

        List<Candidato> candidatos = new ArrayList<>();
        candidatos.add(c1);
        candidatos.add(c2);
        candidatos.add(c3);
        candidatos.add(c4);
        candidatos.add(c5);

        VectorPar v1 = new VectorPar(10);
        v1.agregar(new Par("MNR", 24));
        v1.agregar(new Par("MAS", 30));
        v1.agregar(new Par("NP", 20));
        v1.agregar(new Par("CONDEPA", 15));
        v1.agregar(new Par("ADN", 10));

        VectorPar v2 = new VectorPar(10);
        v2.agregar(new Par("MNR", 20));
        v2.agregar(new Par("MAS", 25));
        v2.agregar(new Par("NP", 30));
        v2.agregar(new Par("CONDEPA", 10));
        v2.agregar(new Par("ADN", 0)); // Added for consistency with
other VPs

        VectorPar v3 = new VectorPar(10);
        v3.agregar(new Par("MNR", 15));
        v3.agregar(new Par("MAS", 20));
        v3.agregar(new Par("NP", 25));
        v3.agregar(new Par("CONDEPA", 30));
        v3.agregar(new Par("ADN", 10));
```

```java
        Mesa m1 = new Mesa(1, v1);
        Mesa m2 = new Mesa(2, v2);
        Mesa m3 = new Mesa(3, v3);

        List<Mesa> mesas = new ArrayList<>();
        mesas.add(m1);
        mesas.add(m2);
        mesas.add(m3);

        OEP oep = new OEP(candidatos, mesas);
    }
}
```

a) Mostrar todos lo datos de la OEP

```
print("-----a) DATOS DE LA OEP-----")
oep.mostrar()
```

```
public class solucion {
    public static void main(String[] args) {
        System.out.println("-----a) DATOS DE LA OEP-----");
        oep.mostrar();
    }
}
```

b) Mostrar la cantidad de votos más baja del candidato del partido X

```
class VectorPar:
    def buscarMenor(self, valor1):
        mini = Par("",999)
        for v in self.vector:
            if(v.getValor1() == valor1):
                if(mini.getValor2() > v.getValor2()):
                    mini = v
        return mini

class OEP:
    def mostrarCantidadVotosMasBaja(self, partido):
        votos = [mesa.getVotos().buscarMenor(partido) for mesa in
self.mesas]
        mini = votos[0]
        for voto in votos:
            if(mini.getValor2()>voto.getValor2()):
                mini = voto
        mini.mostrar()

print("\n-----b) MOSTRAR LA CANTIDAD DE VOTOS MAS BAJA DEL CANDIDATO
DEL PARTIDO X-----")
partido_x = "MNR"
oep.mostrarCantidadVotosMasBaja(partido_x)
```

```java
class VectorPar {
    public Par buscarMenor(String valor1) {
        Par mini = new Par("", 999);
        for (Par v : this.vector) {
            if (v.getValor1().equals(valor1)) {
                if (mini.getValor2() > v.getValor2()) {
                    mini = v;
                }
            }
        }
        return mini;
    }
class OEP {
    public void mostrarCantidadVotosMasBaja(String partido) {
        List<Par> votos = new ArrayList<>();
        for (Mesa mesa : this.mesas) {
            votos.add(mesa.getVotos().buscarMenor(partido));
        }

        Par mini = votos.get(0);
        for (Par voto : votos) {
            if (mini.getValor2() > voto.getValor2()) {
                mini = voto;
            }
        }
        mini.mostrar();
    }
}
public class solucion {
    public static void main(String[] args) {
        System.out.println("\n-----b) MOSTRAR LA CANTIDAD DE VOTOS
MAS BAJA DEL CANDIDATO DEL PARTIDO X-----");
        String partido_x = "MNR";
        oep.mostrarCantidadVotosMasBaja(partido_x);
```

```
    }
}
```

c) Mostrar los candidatos, su partido y la cantidad total de votos que obtuvieron

```
class VectorPar:
    def conseguirVotos(self, partido):
        for v in self.vector:
            if(v.getValor1() == partido):
                return v.getValor2()
        return 0
class OEP:
    def mostrarCandidatosResultados(self):
        for can in self.candidatos:
            votos =
sum([m.getVotos().conseguirVotos(can.getPartidoPolitico()) for m in
self.mesas])
            print(f"{can.getNombre()} {can.getPartidoPolitico()} con:
{votos} votos.")

print("\n-----c) MOSTRAR LOS CANDIDATOS, SU PARTIDO Y LA CANTIDAD
TOTAL DE VOTOS QU OBTUVIERON")
oep.mostrarCandidatosResultados()
```

```
class VectorPar {
    public int conseguirVotos(String partido) {
        for (Par v : this.vector) {
            if (v.getValor1().equals(partido)) {
                return v.getValor2();
            }
        }
        return 0;
    }
}
class OEP {
    public void mostrarCandidatosResultados() {
        for (Candidato can : this.candidatos) {
            int votos = 0;
            for (Mesa m : this.mesas) {
                votos +=
m.getVotos().conseguirVotos(can.getPartidoPolitico());
            }
            System.out.println(can.getNombre() + " " +
can.getPartidoPolitico() + " con: " + votos + " votos.");
        }
    }
public class solucion {
    public static void main(String[] args) {
        System.out.println("\n-----c) MOSTRAR LOS CANDIDATOS, SU
PARTIDO Y LA CANTIDAD TOTAL DE VOTOS QU OBTUVIERON");
        oep.mostrarCandidatosResultados();
```

d) Guardar a/los candidato(s), el partido al que representan y los votos, que obtuvo/obtuvieron

```
class OEP:
    def formatoJson(self):
        data = {}
        i = 1
        for c in self.candidatos:
            data[f"candidato {i}"] ={
                "nombre": c.getNombre(),
                "partido": c.getPartidoPolitico(),
                "votos": self.conseguirVotosCandidatoX(c.getNombre())
            }
            i+=1
        return data
class ArchivoResultado:
    def guardar(self, oep: OEP):
        with open(self.nombreArchivo, 'w') as file:
            data = oep.formatoJson()
            json.dump(data, file, indent=4)


print("\n-----d) GUARDAR LOS DATOS")
archivo = ArchivoResultado("resultados.json")
archivo.guardar(oep)
```

```java
class OEP {
    public String formatoTexto() {
        StringBuilder sb = new StringBuilder();
        int i = 1;
        for (Candidato c : this.candidatos) {
            sb.append("Candidato ").append(i).append(":\n");
            sb.append("  Nombre:
").append(c.getNombre()).append("\n");
            sb.append("  Partido:
").append(c.getPartidoPolitico()).append("\n");
            sb.append("  Votos:
").append(this.conseguirVotosCandidatoX(c.getNombre())).append("\n");
            i++;
        }
        return sb.toString();
    }
}
class ArchivoResultado {
    public void guardar(OEP oep) {
        try (FileWriter fileWriter = new
FileWriter(this.nombreArchivo)) {
            String data = oep.formatoTexto();
            fileWriter.write(data);
            System.out.println("Datos guardados exitosamente en " +
this.nombreArchivo);
        } catch (IOException e) {
            System.err.println("Error al guardar el archivo: " +
e.getMessage());
        }
    }
}
public class solucion {
    public static void main(String[] args) {
        System.out.println("\n-----d) GUARDAR LOS DATOS");
```

```
        ArchivoResultado archivo = new
ArchivoResultado("resultados.txt");
        archivo.guardar(oep);
    }
}
```